

AVP 4주차 세미나

F1tenth 자율주행 캠프 리뷰

MINGYU CHOI

Architecture and Compiler for Embedded System LAB.

School of Electronics Engineering, KNU, KOREA

2026-07-07



목차

- F1tenth 자율주행 캠프 실습
- 앞으로의 HL FMA대회 준비과정 및 교육 커리큘럼 수정

F1tenth라 무엇인가

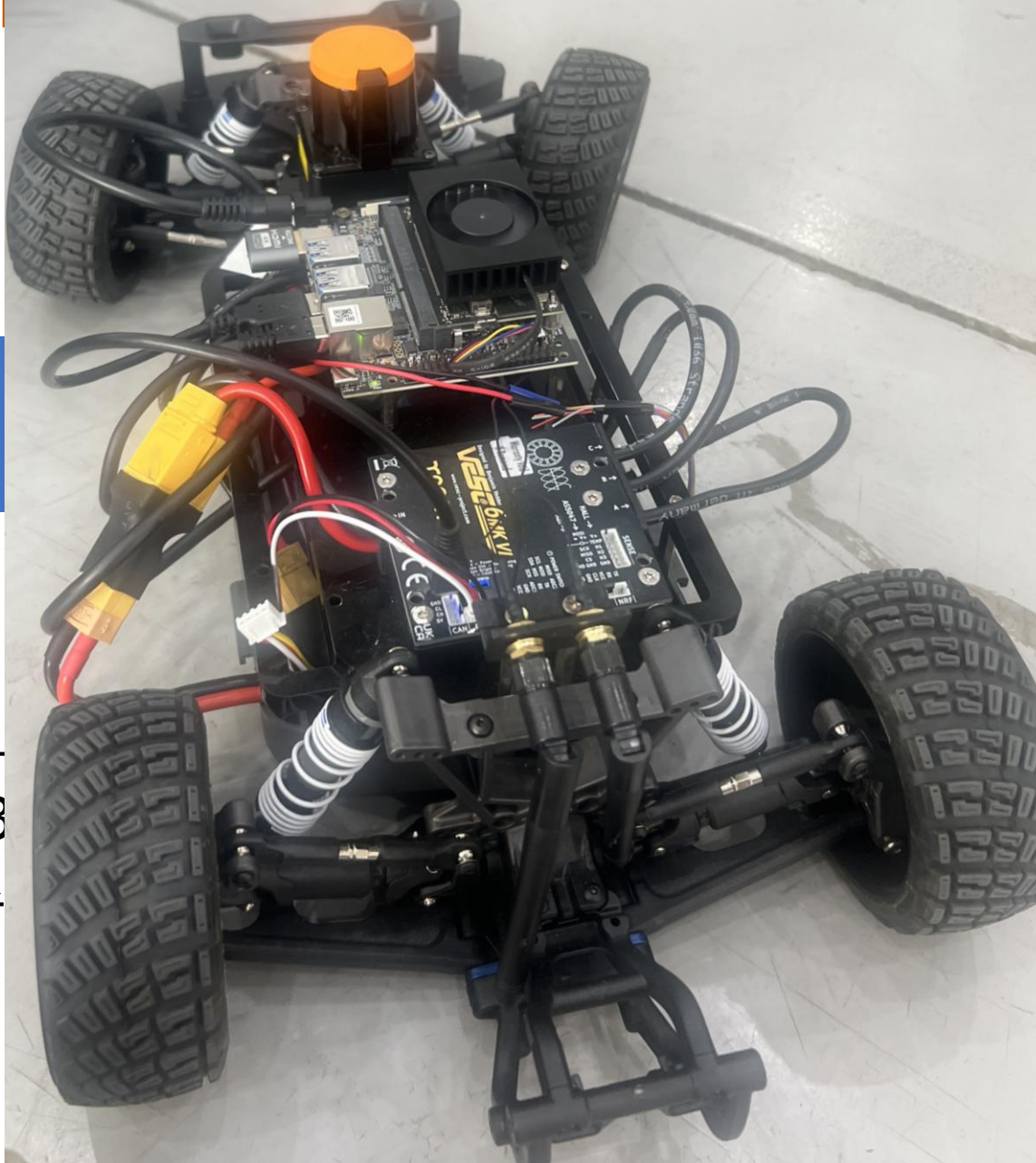
- 1/10 스케

[센서]
LiDAR (Hokuyu
IMU

- ROS2 기반
- LiDAR가 3
- VESC : 속

[동]
ESC 모터 컨트롤러
보 (조향)

주고받는 구조
파악



AEB (Automatic Emergency Braking)

- 앞에 장애물이 있을 때, 충돌까지 남은 시간(TTC)이 임계값 미만이면 자동으로 제동

TTC (Time To Collision) 계산

$TTC = \text{현재 거리}(r) / \text{접근 속도}(dr/dt)$

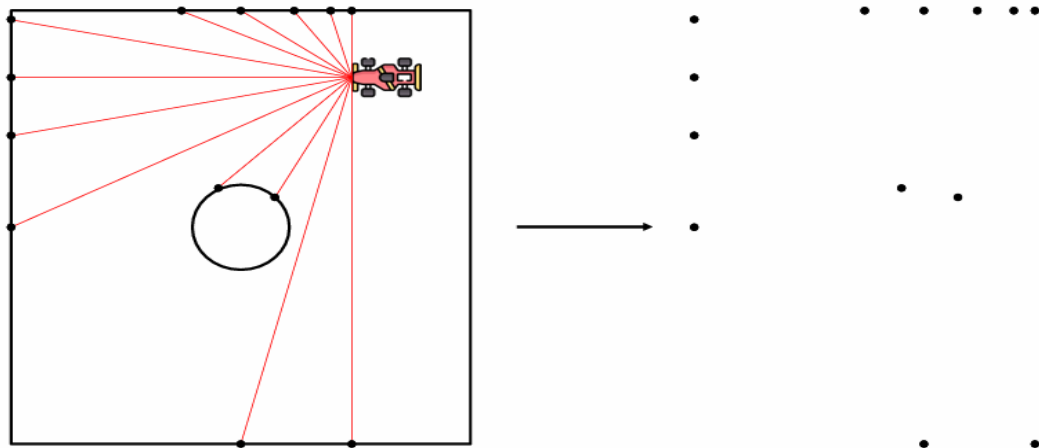
$dr/dt = (r_{\text{현재}} - r_{\text{이전}}) / \Delta t \leftarrow \text{LiDAR 거리 변화율}$

핵심 파라미터

- **TTC_THRESHOLD**: 제동 시작 임계값 (너무 크면 과민, 너무 작으면 늦게 제동)
- 정면 각도 범위: 좁을수록 정밀, 넓을수록 측면 장애물도 반응

SLAM (Simultaneous Localization and Mapping)

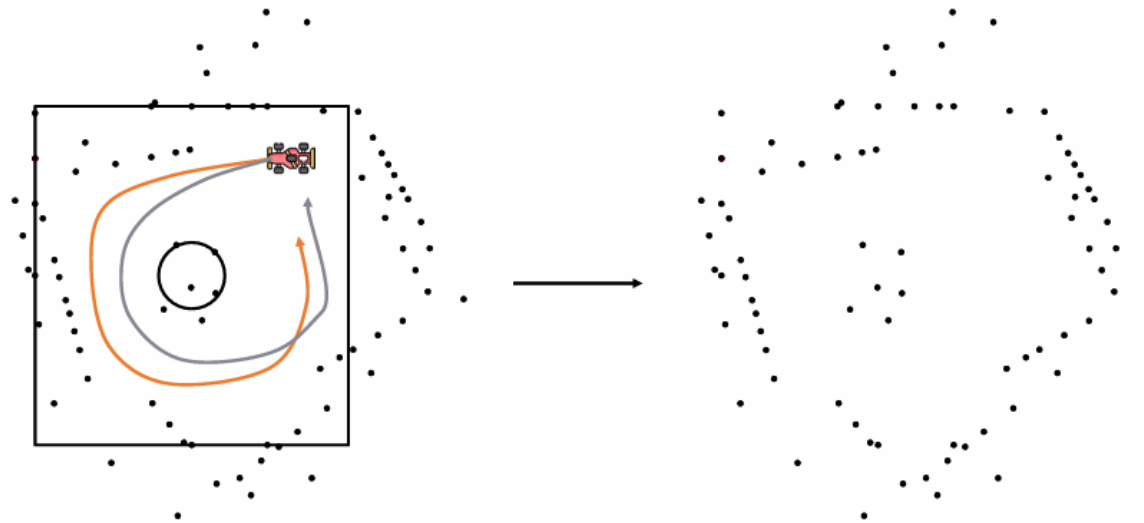
- 지도를 모르는 상태에서 동시에 지도를 만들면서 자신의 위치도 추정한다
- Mapping : LiDAR 스캔 데이터를 누적하여 2D 격자 지도 생성
- Localization : 생성된 지도 위에서 현재 내 위치(x, y, θ) 추정



➔ LiDAR를 이용해 주변에 대한 단편적인 정보를 얻어 지도 생성

SLAM (Simultaneous Localization and Mapping)

- 현실 세계에서는 추정과 실제 차의 위치의 오차 존재

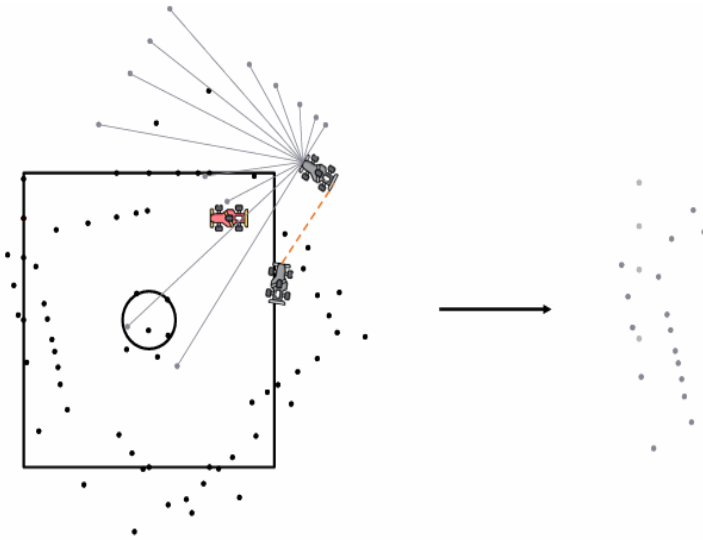


결국 우리가 얻는 맵은 현실과 전혀 다른 정보를 가진다

- ➔ 그래프 SLAM : 위치를 저장하는데에 있어 그래프 구조를 사용
- Loop Closure 기법 : 이미 지나간 장소를 다시 방문했음을 인식하여 지도의 누적 오차를 보정하는 과정

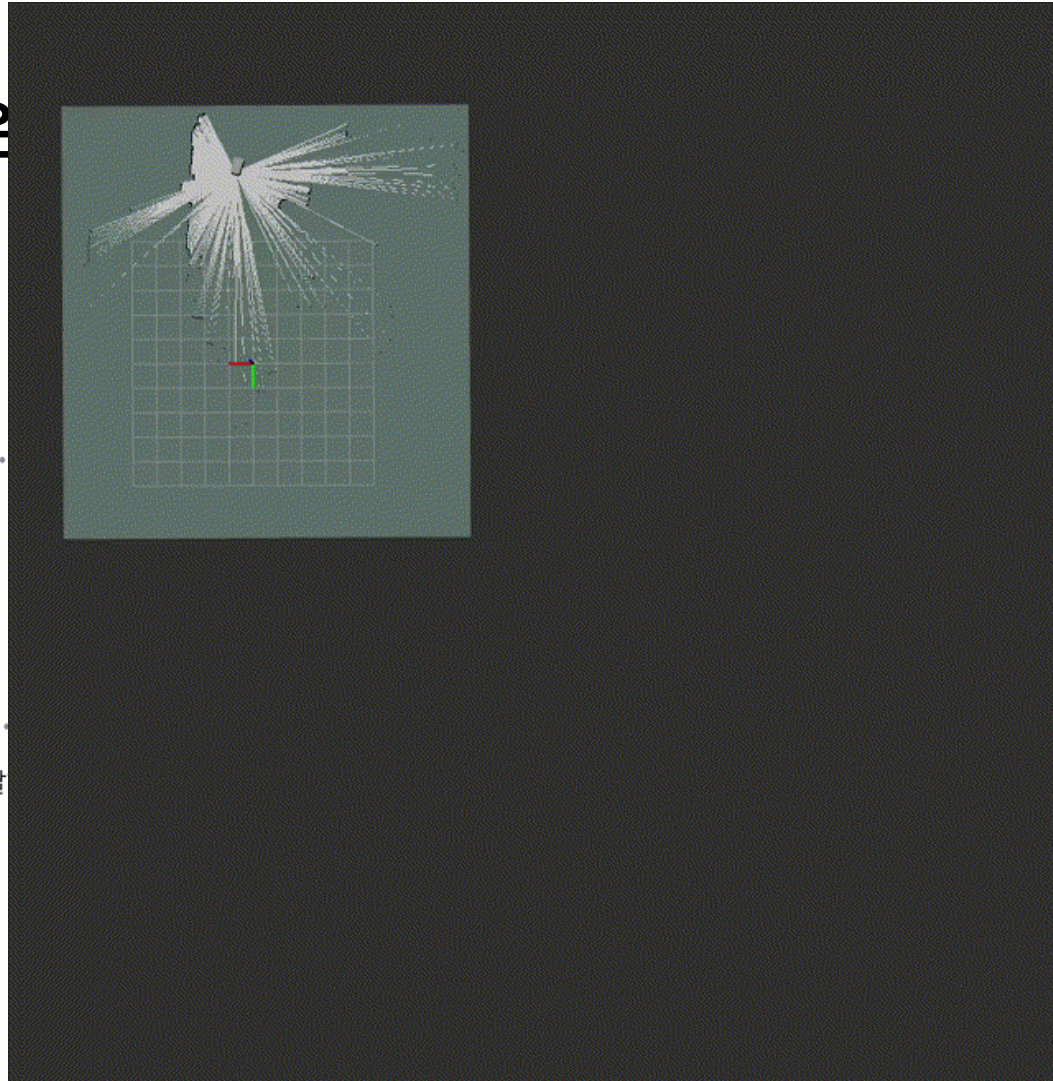
Loop closure

- 지금 보고 있는 장면이
아래와 같다면 두 위치는 같은

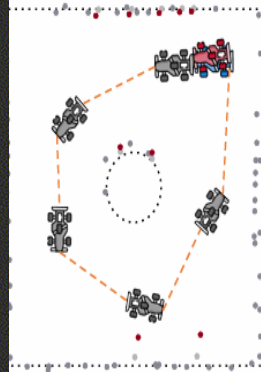


LiDAR 매칭 알고리즘을 통해, 유사한 위치에 도달

➔ slam_toolbox 사용



걸 알



실습과정 (Map 만들기 - slam_toolbox 사용)

1. 패키지 설치

```
sudo apt update
```

```
sudo apt install ros-humble-slam-toolbox
```

```
sudo apt install ros-humble-rviz2
```

2. Launch 파일 실행 (터미널 1)

```
cd ~/f1tenth_ws // 워크스페이스
```

```
source /opt/ros/humble/setup.bash
```

```
source install/setup.bash // 빌드 시 결과물 터미널 환경  
으로
```

```
ros2 launch f1tenth_stack bringup_launch.py
```


실습과정 (Map 만들기 - slam_toolbox 사용)

3. 주행 데이터 기록 (터미널 2)

```
cd ~/f1tenth_ws/src
```

```
source /opt/ros/humble/setup.bash
```

```
source install/setup.bash
```

```
ros2 bag record -a // 파일로 모든 토픽 녹화
```

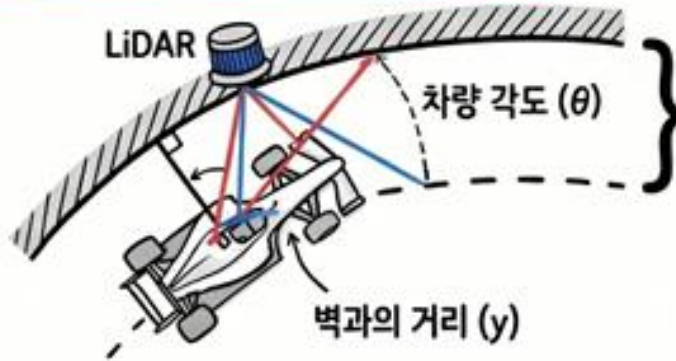
➔ 조이스틱(or 키보드)으로 트랙을 한 바퀴 천천히 주행한 뒤,
Ctrl+C로 기록을 끝내고, 터미널 1의 스택 launch파일도 종료
+ 일정한 속도로 부드럽게, 출발점으로 정확히 되돌아오도록 차를
운전하여 Loop Closure가 걸리게 해야함

4. RVIZ2 실행 (터미널 3) -> 지도 확인

Wall_Follow - 벽 추종

- 양쪽 또는 한쪽 벽과 일정한 거리를 유지하며 주행하고 PID 제어로 거리 오차를 조향각으로 변환

벽 따라가기 알고리즘 적용



궤적 직교 오차 (Cross Track Error - CTE):
목표 경로와 실제 위치 수직 거리

조향각 계산:

$$V_{\theta} = K_p \times e(t) + K_d \frac{de(t)}{dt}$$

(이전 조향각 보정)

제어 계수 튜닝 (Gain Tuning - K_p , K_i , K_d)

튜닝 결과 비교

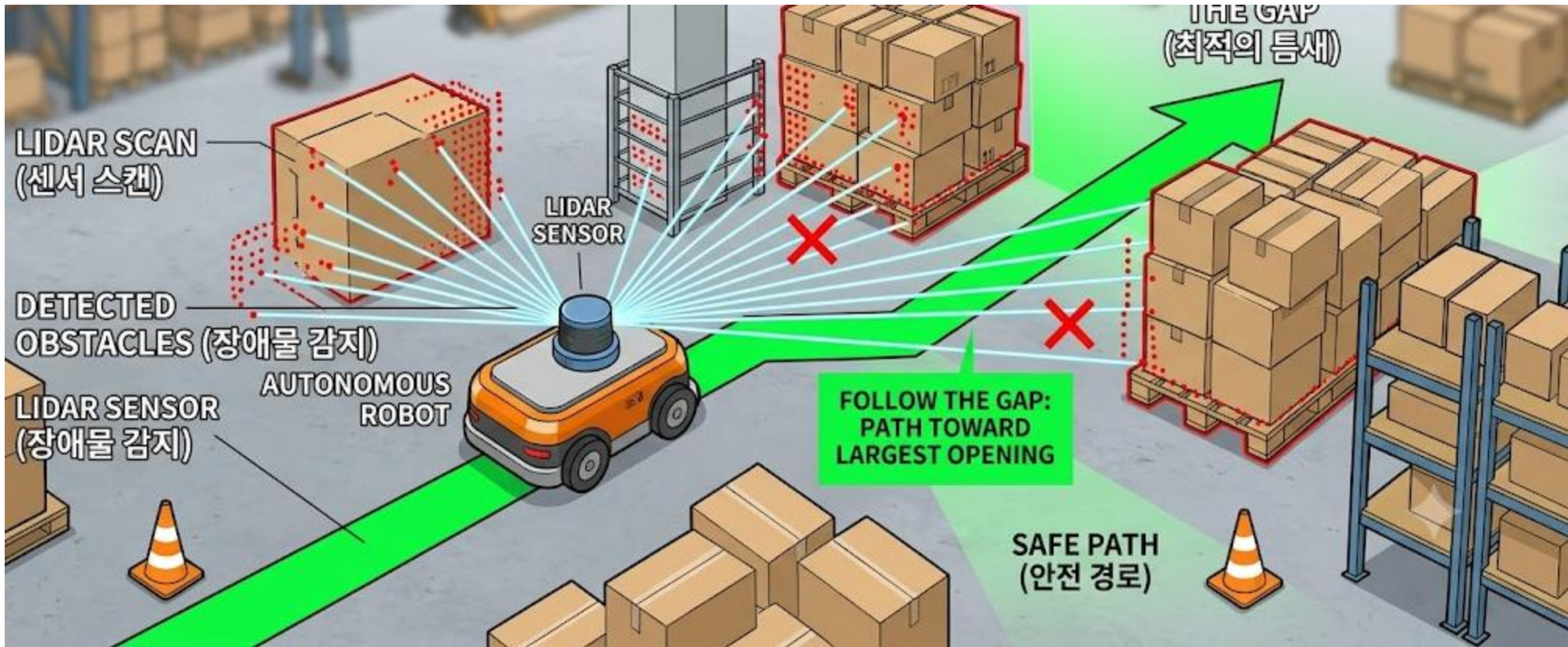
제어 설정	성능 결과	비고
$K_p = 14, K_i = 2, K_d = 0$	느림	오버슈트 발생 가능성 높음
$K_p = 5, K_i = 0, K_d = 0.09$	빠름	비교적 안정적인 주행
$K_p = 14, K_i = 0, K_d = 0.09$	매우 빠름!	가장 이상적인 수렴 성능

성능 특성: 안정적인 행동, 빠른 응답, 부드러운 조향 보장



Follow_the_gap - 장애물 피하기

- LiDAR 스캔에서 가장 넓은 빈 공간(Gap)을 찾아 그 방향으로 돌진하여 장애물이 많은 환경에서 빠르고 직관적으로 장애물 회피



Follow_the_gap - 장애물 피하기



Pure Pursuit – 경로 추종

- 미리 계획된 경로(waypoint) 위에서 차량 앞 일정 거리 (Lookahead distance) 위의 목표점을 향해 곡선 주행

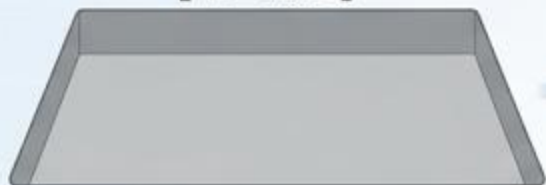
차량의 현재 위치에서 경로 상의 특정 거리만큼 떨어져있는 전방 주시점(Lookahead Point)을 찾고, 현재 위치와 그 지점을 매끄럽게 연결하는 원호(Circular Arc)를 그려 조향각을 계산한다.



그림 4-1. Pure Pursuit 기하 구조: 차량 좌표계($local_x$, $local_y$), lookahead 원(반경 L_d), 목표점, 조향각 δ 의 관계

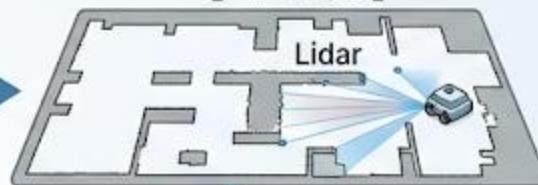
자율 주행 로봇 내비게이션 플로우

[지도 없음]

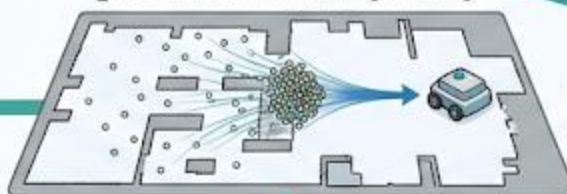


SLAM

[지도 생성]



[Particle Filter (MCL)]



[내 위치 파악]

[장애물 없음]



Pure Pursuit

[좁은 복도]



Wall Follow

[복잡한 환경]



Follow the Gap

HL FMA 대회 준비 전략



[R2R] 무작정 따라하는 ROS2 - 응용과정

[전체 학습 프로그램 보기](#)

- package와 node 생성, topic 구독, 터미널 파라미터 다루기, log 모니터링, rosbag, RQT, ROS2 launch 등 실전에 필요한 개념들을 모아놓은 과정



[R2R] 무작정 따라하는 ROS2 - 심화과정

[전체 학습 프로그램 보기](#)

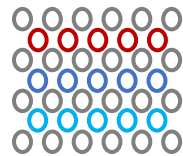
- RVIZ2, IMU 센서, SLAM_toolbox, Nav2 등 심화 기능에 대한 개념 및 실습 과정

HL FMA 대회 전략

- 차선·신호 인식, 장애물 회피 주행, 자동 주차 등 미션 대비
- 전역 경로 – Teach & Repeat + Pure Pursuit
조이스틱으로 이상적 라인을 직접 몰아서 waypoint로 기록하고 pure pursuit로 되밟기
- OpenCV로 차선을 인식하고 라이다를 통해 장애물 검출 전용으로 배치
- Rosbag 전략 로깅으로 데이터를 확보하여 파라미터 튜닝

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)